

SHREE UTTAR GUJARAT BCA COLLEGE

: - INDEX -:

NAME :-	CLASS :- TYBCA
ROLL_NO :-	SUBJECT: - UNIX &Shell Prog.

<u>NO.</u>	<u>Topic</u>	<u>Sign</u>
1.	Practical sheet - 1	
2.	Practical sheet - 2	

Practical Sheet -1

Q: 1 Queries based on Unix basic commands (who, date, cal, touch, tty).

1. WAC (Write a Command) to display number of logging users.

Answer:

`who | wc -l`

Output:

user1 pts/0 2023-08-23 09:00 (192.168.1.1)

user2 pts/1 2023-08-23 10:00 (192.168.1.2)

2. WAC to generate 0(zero) byte file in Unix.

Answer:

`touch empty.txt`

3. WAC to send a message to terminal tty1 using echo command.

Answer:

`echo "Hello, this is a message for tty1" > /dev/tty1`

Output:

Hello, this is a message for tty1

4. WAC to display current time in 12 hours format.

Answer:

`date '+ %l: %M: %S %p'`

Output:

07: 47: 29 PM

5. WAC to display terminal name you are using now.

Answer:

tty

Output:

/dev/pty0

6. WAC to copy file x1 into x2 don't use cp command.

Answer:

cat x1.txt > x2.txt

7. WAC to display current month and year.

Answer:

date '+ %m %Y'

8. WAC to display full calendar of year 1990.

Answer:

cal 1990

9. WAC to display only login name of current user.

Answer:

whoami

Output:

satyam

10. WAC to sets the time and date by admin.

Answer:

date -s "23 AUG 2023 20:04:20"

Output:

Wed Aug 23 20:04:20 IST 2023

11. WAC to run script x1 in background so that its execution continues and User logout from the system.

Answer:

```
./x1.sh &
```

Q:2 Queries based on filter utilities commands (head, tail, cut, paste, sort, tr, cmp, diff, comm, uniq, wc, tee).

1. WAC to count number of characters in first five lines of file x1.

Answer:

```
file:a
```

```
b
```

```
c
```

```
de
```

```
fgh
```

```
ju
```

```
head -5 x1.txt | wc -c
```

Output:

```
18
```

2. WAC to remove a file forcibly which do not have write permission.

Answer:

```
rm -f x1.txt
```

3. WAC to run a utility x1 at 5:00 pm.

Answer:

```
sleep $(date -d '17:00' +%s -u -) && ./x1.txt
```

4. WAC to to open last modified file for editing purpose.

Answer:

```
tail "$(ls -t | head -1)" | less
```

5. WAC to Display only hidden files of working directory.

Answer:

```
ls -a | grep "^."
```

6. WAC to copy a file xyz1 in a parent directory.

Answer:

```
cp "xyz1.txt" "../"
```

or

```
cp "xyz1" "../" | tee copy_log.txt
```

7. WAC to merge the contents of file a.txt, b.txt and c.txt sort them and display the sorted output on screen page by page.

Answer:

```
cat a.txt b.txt c.txt | sort | less
```

Output:

file a

file b

file c

8. WAC to display common lines in x1 and x2.

Answer:

file x1.txt:

This is file x1.

For unix command.

Today is wednesday.

file x2.txt:

This is file x2.

For unix command.

Today is wednesday.

comm -12 x1.txt x2.txt

Output:

comm: file 1 is not in sorted order

comm: file 2 is not in sorted order

Today is wednesday.

comm: input is not in sorted order

9. WAC to remove duplicate lines from a file.

Answer:

file: x1.txt

This is file x1.

For unix command.

For unix command.

Today is wednesday.

uniq x1.txt

Output:

This is file x1.

For unix command.

Today is wednesday.

10. WAC to kill the most recent background process whom neither PID nor job is known.

Answer:

kill %

Q:3 Queries based on simple filter commands (grep, sed).

1. WAC to replace the word computer with computing of file x1.

Answer:

file: x1.txt

I am computer application student.

sed 's/computer/computing/g' x1.txt

Output:

I am computing application student.

2. WAC to display lines starting from 10th line to end of file x1.

Answer:

file x1.txt:

1

2

3

4

5

6

7

8

9

10

11

12

```
sed -n '10,$p' x1.txt
```

Output:

10

11

12

3. WAC to display character from 5th position to 10th position Including both from file x1.

Answer:

file: x1.txt:

this is just an example.

```
sed -r 's/^.{4}(.{6}).*$/\1/' x1.txt
```

Output:

is ju

4. WAC to display number of lines of file x1.

Answer:

file: x1.txt:

this is just an example.

2nd line

```
grep -c "" x1
```

Output:

1

5. WAC to display all blank lines of file x1.

Answer:

```
grep -n '^$' x1.txt
```


6. WAC to Display lines at file x1 that do not begin with (^).

Answer:

file: x1.txt:

this is just

^ an example.

```
grep -v '^\' x1.txt
```

Output:

this is just

7. WAC to count number of blank lines in a file x1.

Answer:

```
grep -c '^$' x1.txt
```

Output:

2

8. WAC to display lines from x1 having exactly three characters.

Answer:

```
grep -w '^...$' x1.txt
```

Output:

Abc

9. WAC to remove all alphabets from the file x1.

Answer:

```
sed 's/[[:alpha:]]//g' x1.txt > x1.txt
```

10. WAC to delete lines matching pattern “surat”.

Answer:

```
sed '/surat/d' x1.txt
```

Practical Sheet -2

1. Write script, using case statement to perform basic math operations (+,-,*,/,%).

Answer:

```
clear

read -p "Enter the first number: " n1
read -p "Enter the second number: " n2
read -p "Enter the operators according to operation (+, -, *, /, %) and e to exit: " operator

case $operator in
    +)
        echo "Addition of $n1 and $n2 is `expr $n1 + $n2`"
        ;;
    -)
        echo "Substraction of $n1 and $n2 is `expr $n1 - $n2`"
        ;;
    \*)
        echo "Multiplication of $n1 and $n2 is `expr $n1 \* $n2`"
        ;;
    /)
        echo "division of $n1 and $n2 is `expr $n1 / $n2`"
        ;;
    %)
        echo "Remainder of division of $n1 and $n2 is `expr $n1 % $n2`"
        ;;
    e)
        exit
        ;;
    *)
        echo "Invalid operator"
```

```
;;  
esac
```

output:

Enter the first number: 4

Enter the second number: 15

Enter the operators according to operation (+, -, *, /, %) and e to exit: +

Addition of 4 and 15 is 19

Enter the first number: 4

Enter the second number: 6

Enter the operators according to operation (+, -, *, /, %) and e to exit: -

Substraction of 4 and 6 is -2

Enter the first number: 8

Enter the second number: 12

Enter the operators according to operation (+, -, *, /, %) and e to exit: *

Multiplication of 8 and 12 is 96

Enter the first number: 8

Enter the second number: 4

Enter the operators according to operation (+, -, *, /, %) and e to exit: /

division of 8 and 4 is 2

Enter the first number: 8

Enter the second number: 4

Enter the operators according to operation (+, -, *, /, %) and e to exit: %

Remainder of division of 8 and 4 is 0

Enter the first number: 5

Enter the second number: 3

Enter the operators according to operation (+, -, *, /, %) and e to exit: e

2. Write a script to display the according to the time like good morning, good afternoon, good evening and good night.

Answer:

```
hour=$(date '+ %H')
if [ $hour -ge 5 ] && [ $hour -lt 12 ]; then
    echo "Good morning"
elif [ $hour -ge 12 ] && [ $hour -lt 17 ]; then
    echo "Good afternoon"
elif [ $hour -ge 17 ] && [ $hour -lt 20 ]; then
    echo "Good evening"
else
    echo "Good night"
fi
```

output:

```
$ sh unix.sh
```

Good afternoon

3. Write script to check inputted file is regular file , directory or does not exist ..

Answer:

```
read -p "Enter file name: " f
if test -e $f ; then
    if test -f $f ; then
        echo "It is a regular file"
    elif test -d $f ; then
```

```
        echo "It is a directory"
    fi
else
    echo "$f does not exist"
fi
```

Output:

Enter file name: f1.txt

f1.txt does not exist

Enter file name: unix.sh

It is a regular file

4. Write a script which enters username s& password & check that if the username = sugc & password=12345 then display the valid user message. Otherwise invalid user. [Script gives maximum 3 attempts to the user.]

Answer:

```
attempts=0
while [ $attempts -lt 3 ]
do
    read -p "Enter username: " username
    read -p "Enter password: " password
    if [ $username = "sugc" -a $password = "12345" ] ; then
        echo "Valid User"
        exit
    else
        echo "Invalid User"
        attempts=`expr $attempts + 1`
        echo "Remaining attempts: `expr 3 - $attempts`"
```

```
fi  
done
```

Output:

```
$ sh unix.sh  
Enter username: a  
Enter password: 3  
Invalid User  
Remaining attempts: 2  
Enter username: s  
Enter password: 12345  
Invalid User  
Remaining attempts: 1  
Enter username: sugc  
Enter password: pass  
Invalid User  
Remaining attempts: 0  
  
Enter username: sugc  
Enter password: 12345  
Valid User
```

5. Accept a string from terminal and echo suitable message if it does not have at least 10 characters.

Answer:

```
read -p "Enter a String: " str  
char_count=$(echo -n $str | wc -c)  
if [ $char_count -lt 10 ] ; then  
    echo "The input string does not have at least 10 characters."  
else
```

```
    echo "The input string has at least 10 characters."  
fi
```

Output:

```
$ sh unix.sh  
Enter a String: suyghsu  
The input string does not have at least 10 characters.
```

6. Write a script to delete all vowels from particular string.

Answer:

```
read -p "Enter a string: " str  
str_without_vowels=$(echo $str | sed 's/[aeiouAEIOU]//g')  
echo "String without vowels: $str_without_vowels"
```

Output:

```
$ sh unix.sh  
Enter a string: unix  
String without vowels: nx
```

7. Write a script to accept a number from user until he enters 0 & find sum of all that numbers.

Answer:

```
sum=0  
reached_zero="no"  
while [ $reached_zero = "no" ] ; do  
    read -p "Enter a number (zero to exit): " input  
    if [ $input -eq 0 ] ; then  
        reached_zero="yes"  
        break  
    fi
```



```
sum=`expr $sum + $input`  
done  
echo "Sum of entered numbers: $sum"
```

Output:

```
$ sh unix.sh  
Enter a number (zero to exit): 1  
Enter a number (zero to exit): 2  
Enter a number (zero to exit): 3  
Enter a number (zero to exit): 0  
Sum of entered numbers: 6
```

8. Write an awk script to print each odd line twice and even line thrice.

Answer:

```
awk '{  
    if (NR % 2 == 1) {  
        print  
        print  
    } else {  
        print  
        print  
        print  
    }  
}' f1.txt
```

Output:

```
$ sh unix.sh  
first line  
first line
```

second line

second line

second line

third line

third line

f1.txt:

first line

second line

third line

9. Check whether file is empty or not. And if file name doesn't exist than print appropriate msg.

Answer:

```
read -p "Enter file name: " filename
```

```
if test -e $filename ; then
```

```
    if test -s $filename ; then
```

```
        echo "$filename is not empty"
```

```
    else
```

```
        echo "$filename is empty"
```

```
    fi
```

```
else
```

```
    echo "$filename doesn't exists"
```

```
fi
```

Output:

```
$ sh unix.sh
```

```
Enter file name: f1.txt
```

```
f1.txt is not empty
```

10. Write a shell script which takes input of file name and prints first 10 lines of that file. File name is to be passed as command line argument. If argument is not passed then any 'C' program from the current directory is to be selected

Answer:

```
if [ $# -eq 0 ]; then
    c_program=$(ls *.c | head -10)
    if [ -z $c_program ]; then
        echo "No, C program found in current directory."
        exit 1
    fi
    filename=$c_program
else
    filename=$1
fi
if [ -f "$filename" ]; then
    head -10 "$filename"
else
    echo "File '$filename' not found."
fi
```

Output:

```
$ sh unix.sh f1.txt
```

first line

second line

third line

```
$ sh unix.sh
```

```
#include<stdio.h>
```

```
#include<conio.h>

void main()
{
    int a[5][5],b[5][5],c[5][5],i,j;
    clrscr();
    for (i=0;i<2;i++)
    {
        for (j=0;j<2;j++)
        {
```